

The engineering ruleset workflow

How a repository is set up, how a branch becomes a merge, and how the ticket moves itself along the way.

Two things are running in a governed repository.

Guardrails — GitHub rulesets that refuse pushes, branch names and merges server-side, for everyone, whether or not they have the plugin installed. And **automation** — generated workflows that move your ticket through its statuses as you work, and that review your pull request before a human does.

Both come from one file, `ruleset-config.json`. Nothing is configured twice.

Part 1	Setting a repository up	1
Part 2	What now exists — the rulesets and workflows	5
Part 3	The developer loop, in eight steps	6
Part 4	When something blocks you	10
Part 5	Admin operations	11

PART 1

Setting a repository up

Done once, by someone with **admin** on the repository. If you are joining a repo that already works, skip to [Part 3](#).

Prerequisites

NEED	WHY
<code>gh</code> CLI, authenticated	the plugin authenticates through it — <code>gh auth login</code>
Admin on the target repo	rulesets are an admin-scoped API
Node 18+	the CLI runs on it; no install, no build step
ClickUp or Jira	the ticket is what the scope check reads and the sync moves
An OpenRouter API key	both AI checks call it

CHECK THIS BEFORE ANYTHING ELSE

Your tracker must already have the statuses the pipeline writes: `to do` → `in progress` → `dev` → `QA` → `done`. If your board names them differently, change `taskSync.environmentStatuses` — never the generated workflow, which is overwritten on every sync. A status the tracker does not have means that stage silently never lands on ClickUp, and fails the job on Jira.

Run the plan, then apply

```
/enforce-rules --repo owner/name
```

The plan **writes nothing**. Read it before applying — it lists every ruleset it would create, every rule it had to weaken for this repository, and every existing ruleset it will leave alone.

On a repository with no rulesets yet you are asked two questions: **which tracker** (ClickUp, Jira, or none) and **which environments**. It starts at `dev` alone; `test` and `prod` are added on request and arrive with their own policy attached.

Set the credentials

The plugin never sees a token. On your own terminal it hands off to `gh`, which prompts with hidden input and uploads the encrypted value directly.

Every value below can be generated from the link in its row — no hunting through a settings tree. Hand the relevant table to whoever holds the account.

IF YOU USE CLICKUP

VALUE	DESCRIPTION	WHERE TO GET IT	EXAMPLE
CLICKUP_TOKEN SECRET	Personal API token the workflow uses to read and move the task.	app.clickup.com/settings/apps → <i>API Token</i> → Generate , then copy it.	pk_12345678_ABCDEFGH...
CLICKUP_TEAM_ID SECRET · OPTIONAL	Workspace id. Needed only if your workspace uses Custom Task IDs — it switches the sync to the custom-id endpoint.	Open any page in the workspace; it is the number in the URL — <code>app.clickup.com/9013554321/home</code> .	9013554321

IF YOU USE JIRA

VALUE	DESCRIPTION	WHERE TO GET IT	EXAMPLE
JIRA_API_TOKEN SECRET	Authentication token generated from your Atlassian account.	id.atlassian.com/manage-profile/security/api-tokens → <i>Create API token</i> → label it → copy the value.	ATATT3xFfGF0...
JIRA_BASE_URL VARIABLE	Root web address of your Jira Cloud site.	Look at the browser bar on any Jira page. Keep only the origin — drop any path or trailing slash after <code>.net</code> .	<code>https://your-company.atlassian.net</code>
JIRA_EMAIL VARIABLE	Email of the Atlassian account that created the token.	id.atlassian.com/manage-profile/profile-and-visibility , or your avatar → <i>Profile</i> .	you@example.com

EITHER WAY

VALUE	DESCRIPTION	WHERE TO GET IT	EXAMPLE
OPENROUTER_API_KEY SECRET	Key the scope check and PR-Agent both use to call the model.	openrouter.ai/keys → Create Key → name it → copy it once; it is not shown again.	sk-or-v1-...

`JIRA_BASE_URL` and `JIRA_EMAIL` are **variables**, not secrets — they are not sensitive, so they are typed in the clear.

```
gh secret set OPENROUTER_API_KEY --repo owner/name # prompts, hidden
gh variable set JIRA_BASE_URL --repo owner/name # typed in the clear
```

NEVER PASTE A TOKEN ANYWHERE BUT THAT PROMPT

Not into a chat, not onto a command line, not into a file. Anything pasted into a chat must be treated as exposed — revoke it and issue a new one. The hidden prompt is the only safe route, and the CLI refuses arguments that look like tokens for exactly this reason.

`OPENROUTER_API_KEY` is not optional. The scope check is a **required** status check configured to block on its own errors, so a missing or invalid key stops every pull request. That is deliberate: a check that cannot run must not look like a check that passed.

Then apply:

```
/enforce-rules --repo owner/name --apply
```

In order: environment branches are created, the workflow files are committed, then the rulesets go up. That order is forced — a ruleset requiring a pull request for `dev` also refuses the push that *creates* `dev`. On a repo already synced, the blocking rulesets are switched off for the few seconds this takes, then restored. The plan says so first.

AFTER APPLYING

`main` requires a pull request — for every actor, including the repository owner. There are no bypass actors.

What now exists

The guardrails

RULESET	COVERS	REFUSES
Pull Request Compulsion	main + every environment	direct pushes, deletion, force-push. A PR is required (0 approvals)
PR-SCOPE-CHECK	every environment	merging until the scope-check status passes
TRIVY-SECURITY	every environment	merging until the trivy-security scan passes
team-only-reviewer	prod, when added	merging without 1 approval from the reviewer team, or a code owner
Enforce Branch Nomenclature	everything else	creating a branch not shaped <prefix>/<task-id>/<description>

The automation

FILE	FIRES WHEN	DOES
clickup-sync.yml or jira-sync.yml	you push a work branch; a PR merges into an environment	moves the ticket forward
pr-scope-check.yml	a PR targets any environment	asks an AI whether the diff matches the ticket
scripts/scope-check.mjs	—	the logic that workflow runs
pr-agent.yml	a PR opens, or you comment /review	description, review, inline suggestions, security gate
trivy-security.yml	a PR targets any environment	scans for vulnerable dependencies, hardcoded secrets and IaC misconfigurations
scripts/trivy-report.mjs	—	applies the threshold and posts the verdict

All four live under `.github/` and are **regenerated on every sync**. Edit the config, not the workflow.

The developer loop

1 Take a ticket, copy its id

ClickUp	86d3bzhgq	short alphanumeric
Jira	PROJ-123	

2 Create the branch

```
git switch -c feature/86d3bzhgq/checkout-redirect
```

The shape is `<prefix>/<task-id>/<description>`, and all three segments are required.

created feature/86d3bzhgq/checkout-redirect

created bugfix/PROJ-123/null-on-empty-cart

created feature/86d3bzhgq/fix/retry the description may contain slashes

refused feature/86d3bzhgq no description

refused feature/checkout-redirect only two segments

refused wip/86d3bzhgq/thing wip is not an allowed prefix

Allowed prefixes: `feature`, `bugfix`, `hotfix`, `docs`, `chore`. A refused branch fails at *creation*, when you first push it — `push declined due to repository rule violations`. Nothing is lost; rename and push again with `git branch -m`.

WHAT THE RULE CAN AND CANNOT CHECK

A ref pattern can enforce the *shape* — three segments, known prefix — but it cannot tell a real ticket id from any other word. That check happens at pull-request time, where the tracker can actually be asked. The two together are the guarantee.

3 Push — the card moves to *in progress*

```
git push -u origin feature/86d3bzhgq/checkout-redirect
```

That push alone takes the ticket out of to-do. No merge, no pull request, no dragging a card across a board.

Every later push runs the job too and does nothing — the pipeline only moves forward, so a ticket already at *in progress* or past it stays exactly where it is. Recognised to-do spellings: `to do`, `todo`, `open`, `backlog`, `pending`.

4

Open a pull request into `dev`

```
gh pr create --base dev \  
  --title "Checkout redirect" \  
  --body "https://app.clickup.com/t/86d3bzhgq"
```

Base is `dev`; head is your feature branch. The ticket link in the body is the most reliable way for the scope check to find the ticket — the branch name works too, and so does an id in the title.

5

The checks run

PR-Agent writes a summary, posts a review with file and line references, and leaves inline suggestions. It also labels the PR when it finds a security concern — and a second job reads that label and **fails the check**, blocking the merge until it is resolved. Comment `/review`, `/describe` or `/improve` to run it again.

The scope check reads the ticket, compares it against your diff, and posts one verdict:

VERDICT	RESULT
<code>in scope</code>	check passes, merge unblocked
<code>out of scope</code>	check fails and the pull request is closed automatically
<code>no ticket</code>	check fails — link the ticket and push again
<code>errored</code>	check fails — it will not pass on an error

Trivy scans for vulnerable dependencies, hardcoded secrets and IaC misconfigurations, and posts one verdict comment. It needs no credentials. What blocks you is a **severity threshold**, applied only to the files your pull request touched:

<code>blocked</code>	CRITICAL secret in a file you changed
<code>blocked</code>	CRITICAL CVE your PR introduced
<code>reported</code>	CRITICAL CVE with no released fix nothing you could do would clear it
<code>reported</code>	HIGH misconfiguration

`reported` a finding in a file you did not touch

A blocked pull request gets a table naming each finding — severity, file and line, identifier and the fix — plus `::error` annotations on the offending lines. Fix them, or take those files out of the pull request, and push again.

OUT OF SCOPE CLOSES YOUR PULL REQUEST

This is the behaviour most worth knowing before it happens to you. Either strip the unrelated changes into their own branch and ticket, or widen the ticket's description to cover the work — then open a new PR.

Incidental changes are fine — imports, types, small config, the tests that make your change work. What gets flagged is a genuinely unrelated edit riding along: a drive-by refactor, an unrelated dependency bump, a fix for something the ticket never mentions.

6 Merge — the card moves to *dev*

No approval is needed for `dev`; the pull request itself is the requirement.

7 Promote to `test` — the card moves to *QA*

MERGE THE FEATURE BRANCH, NOT DEV

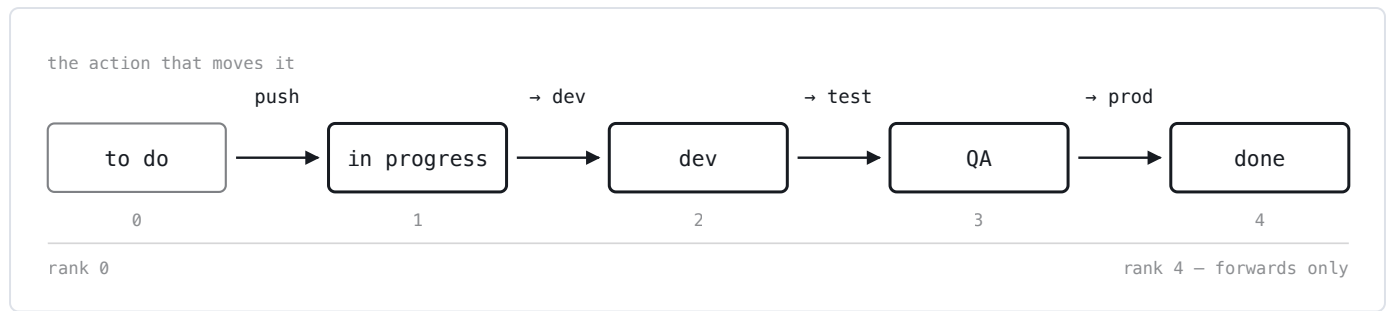
The ticket id is read from the *head branch name* of the merged pull request. `dev` carries no task id, so a `dev → test` promotion moves no card at all — the job logs `No task id in 'dev'; nothing to sync` and exits cleanly. It looks like success and does nothing.

```
gh pr create --base test --head feature/86d3bzhgq/checkout-redirect
```

The scope check runs here too — every environment is covered.

8 Promote to `prod` — the card moves to *done*

Same again with `--base prod`. If `prod` is configured, this one also needs 1 approval from the reviewer team, or from a code owner.



The first arrow is a *push* of the feature branch; the other three are *merges of that same feature branch* into each environment. Rank is why the order holds: a ticket only ever moves to a higher number, so merging an old branch into `dev` can never pull a finished ticket back — and a status nobody declared, like `blocked`, is left alone rather than guessed at.

When something blocks you

WHAT YOU SEE	WHAT IT MEANS	FIX
<code>push declined due to repository rule violations</code> on a new branch	the name is not <code><prefix>/<task-id>/<description></code>	<code>git branch -m</code> to a valid name, push again
Scope check: <i>no ticket linked</i>	nothing in the branch, title or body resolved to a ticket	paste the ticket URL in the PR body, push again
Scope check failed, PR closed	the AI judged part of the diff unrelated to the ticket	split the unrelated work out, or widen the ticket, then open a new PR
Scope check: <i>couldn't complete</i>	usually <code>OPENROUTER_API_KEY</code> missing, invalid, or out of credit	an admin re-sets the secret; it blocks by design until then
Security gate failed	PR-Agent flagged a security concern	read its review comment, fix, push — the check re-runs
Trivy check failed	a <code>CRITICAL</code> finding in a file this PR changed	the comment names the file, line and fix; resolve it, or move that file out of this PR
Trivy flagged a CVE with no fix	upstream has published no patched version	reported, not blocking — if it <i>is</i> blocking, someone set <code>ignoreUnfixed: false</code>
Merge button greyed out on <code>prod</code>	1 approval from the reviewer team is outstanding	request review from the team or a code owner
Card didn't move on merge	the head branch carried no id (a <code>dev</code> → <code>test</code> promotion), or the status name does not exist in the tracker	merge the feature branch itself; check <code>environmentStatuses</code> against your board
Card didn't move on push	branch prefix is not in <code>allowedPrefixes</code> , or the tracker token is missing	the workflow run log says which
<code>⚠ THIS RULESET IS BLOCKING MERGES</code>	a leftover ruleset the config no longer manages demands a review this repo cannot supply	delete or edit it in Settings → Rules

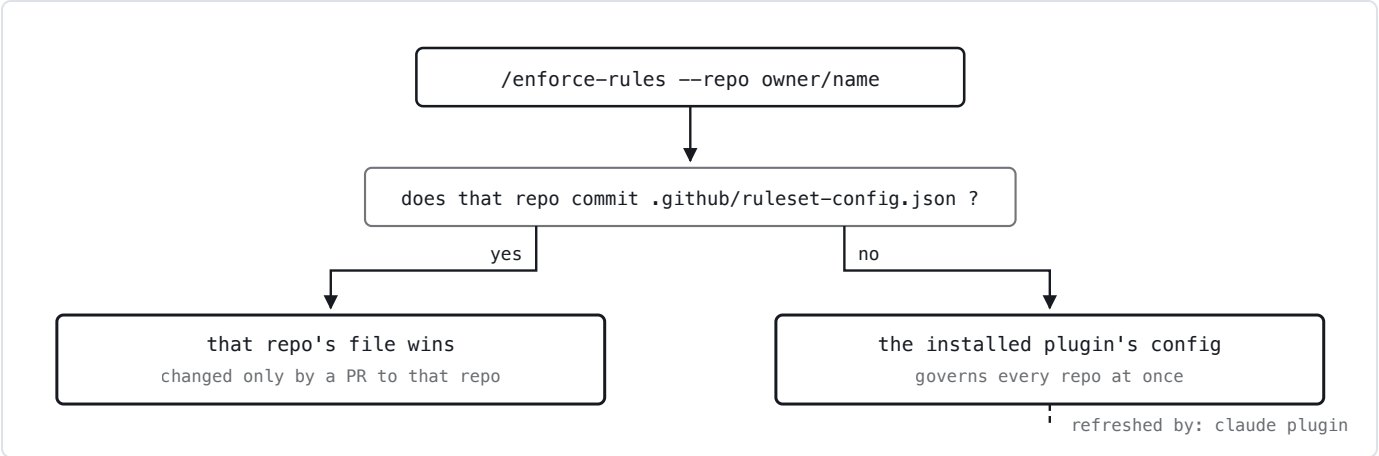
Two that catch admins

“APPLY ... IN THAT FILE VIA A PULL REQUEST, THEN RE-RUN”

The repository commits its own `.github/ruleset-config.json`, and that file **wins** over the plugin's. Policy for that repo can then only change by a pull request to that repo. To govern it centrally instead, delete the override and the plugin's config takes over again.

PLUGIN CHANGES THAT DON'T TAKE EFFECT

`/enforce-rules` runs the **installed** copy of the plugin, not your working tree. Editing a checkout on your Desktop changes nothing until the change reaches the plugin repo's default branch and the install is refreshed with `claude plugin marketplace update`. This is the single most common reason a policy change “didn't apply”.



Which config actually governs a repository — and, on the right, the refresh step that decides whether an edit to the plugin has reached the machine running the command.

	CENTRAL — NO OVERRIDE	PER-REPO — COMMITTED OVERRIDE
Change policy	edit the plugin, refresh, re-sync	a pull request to that repository
<code>--env</code> from the CLI	works	refused
Rolls out to	every repo at once	that repo only

Admin operations

Add an environment

```
/enforce-rules --repo owner/name --env staging --apply
```

It joins the pull-request requirement, gets the scope check, is excluded from the branch-naming rule, and becomes a task-sync stage — all from the same list.

Declaration order is the pipeline order, so an environment added later lands last; reorder `environments` if that is wrong for your real pipeline. An environment with no entry in `environmentStatuses` maps to a status of its own name.

Switch tracker

`--provider jira` rewrites the sync workflow and plans a `DELETE` of the other provider's — leaving it behind would have both trackers moving tickets on every merge.

Turn a check off

Set `prChecks.scopeCheck.enabled` or `prAgent.enabled` to `false`; the next sync removes the file — but only if the plugin wrote it. A hand-written workflow at the same path is never deleted.

Loosen the branch shape

`requireDescription: false` permits the bare `<prefix>/<task-id>` form again. Setting `taskIdPrefix` (say `"CU-"`) narrows the id segment to that marker.

QUICK REFERENCE

```
# start work
git switch -c feature/86d3bzhgq/checkout-redirect
git push -u origin feature/86d3bzhgq/checkout-redirect # → card: in progress

# ship it — always the feature branch, never dev → test
gh pr create --base dev --head feature/86d3bzhgq/checkout-redirect # merge → card: dev
gh pr create --base test --head feature/86d3bzhgq/checkout-redirect # merge → card: QA
gh pr create --base prod --head feature/86d3bzhgq/checkout-redirect # merge → card: done
```

BRANCH SHAPE	<prefix>/<task-id>/<description>
PREFIXES	feature bugfix hotfix docs chore
CARD MOVES ON	a push, and every merge into an environment
DIRECTION	forwards only, never backwards
PROMOTION	merge the feature branch into each environment
OUT OF SCOPE	closes the pull request
TRIVY	CRITICAL in a file you changed blocks; everything else reports
PROD	1 approval from the reviewer team

Generated from `ruleset-config.json` at plugin 1.13.0. The canonical copy of this document lives at `docs/WORKFLOW.md` in the plugin repository — if the two disagree, the config is the truth and this page is stale.